

2d Array

r, c — integer.

data-type arr_name [] [] = new data-type [r] [c] ;

int A [] [] = new int [2] [3] ;

A.

	0	1	2
0			
1			

Print 2d array

```
int r=scn.nextInt();
int c=scn.nextInt();
int A[][]=new int[r][c];
for(int i=0;i<r;i++){
    for(int j=0;j<c;j++){
        A[i][j]=scn.nextInt();
    }
}
```

```
for(int i=0;i<r;i++){
    for(int j=0;j<c;j++){
        System.out.print(A[i][j]+" ");
    }
    System.out.println();
}
```

Wave Traversal -1

A=

	0	1	2	3	4	5
0	11	21	31	41	51	61
1	12	22	32	42	52	62
2	15	25	35	45	55	65
3	14	24	34	44	54	64
4	16	26	36	46	56	66
5	13	23	33	43	53	63

00	51
10	41
20	31
30	21
40	11
50	01

Even c / odd c
($r=0, n-1$) ($r=n-1, 0$)

```
static void waveTraversal_1(int A[][]){
    //code here
    for(int j=0;j<A[0].length;j++){
        if(j%2==0){
            for(int i=0;i<A.length;i++){//top to bottom
                System.out.print(A[i][j]+" ");
            }
        }else{
            for(int i=A.length-1;i>=0;i--){//bottom to top
                System.out.print(A[i][j]+" ");
            }
        }
    }
}
```

Wave Traversal - 2

$A =$

	0	1	2	3	4	5
0	11	21	31	41	51	61
1	12	22	32	42	52	62
2	15	25	35	45	55	65
3	14	24	34	44	54	64
4	16	26	36	46	56	66
5	13	23	33	43	53	63

```
static void waveTraversal_2(int A[][]){
    //code here
    for(int j=0;j<A[0].length;j++){
        if(j%2==1){
            for(int i=0;i<A.length;i++){//top to bottom
                System.out.print(A[i][j]+" ");
            }
        }else{
            for(int i=A.length-1;i>=0;i--){//bottom to top
                System.out.print(A[i][j]+" ");
            }
        }
    }
}
```

C - Even ($r = n-1, 0$)
C - odd ($r = 0, n-1$)

$A =$

	0	1	2	3	4	5
0	11	21	31	41	51	61
1	12	22	32	42	52	62
2	15	25	35	45	55	65
3	14	24	34	44	54	64
4	16	26	36	46	56	66
5	13	23	33	43	53	63

$A =$

	0	1	2	3	4	5
0	11	21	31	41	51	61
1	12	22	32	42	52	62
2	15	25	35	45	55	65
3	14	24	34	44	54	64
4	16	26	36	46	56	66
5	13	23	33	43	53	63

Leetcode 1572 Matrix Diagonal Sum

$A =$

	0	1	2	3	4
0	11	21	31	41	51
1	12	22	32	42	52
2	15	25	35	45	55
3	14	24	34	44	54
4	16	26	36	46	56

11 + 22 + 35 + 44 + 56 +
61 + 42 + 24 + 16

```
class Solution {
    public int diagonalSum(int[][] A) {
        int n=A.length;
        int sum=0,c1=0,c2=n-1;
        for(int r=0;r<n;r++){
            if(c1==c2){
                sum=sum+A[r][c1];
            }else{
                sum=sum+A[r][c1]+A[r][c2];
            }
            c1++;
            c2--;
        }
        return sum;
    }
}
```

Leetcode 74 - Search row-col sorted matrix

	0	1	2	3
0	1	3	5	7
1	10	11	16	20
2	23	30	34	60

T = 23

```
public boolean searchMatrix(int[][] A, int T) {
    int r=0, c=A[0].length-1;
    while(r<A.length && c>=0){
        if(T>A[r][c]){
            r++;
        }else if(T<A[r][c]){
            c--;
        }else{
            return true;
        }
    }
    return false;
}
```

Transpose of the matrix

A

	0	1	2
0	1	2	3
1	4	5	6

2x3

B

	0	1
0	1	4
1	2	5
2	3	6

3x2

```
class Solution {
    public int[][] transpose(int[][] A) {
        int r=A.length;
        int c=A[0].length;
        int B[][]=new int[c][r];
        for(int i=0;i<B.length;i++){
            for(int j=0;j<B[0].length;j++){
                B[i][j]=A[j][i];
            }
        }
        return B;
    }
}
```

$B[0][0] = A[0][0]$
 $B[0][1] = A[1][0]$
 $B[1][0] = A[0][1]$
 $B[1][1] = A[1][1]$

$B[2][0] = A[0][2]$
 $B[2][1] = A[1][2]$
 $B[i][j] = A[j][i];$

ArrayList

- used to store the group of values with same data type
- dynamic array.
- continuous memory allocation.

Syntax

`ArrayList<data-type> al-name = new ArrayList<>();`

`int` — Integer

`char` — Character

`float` — Float

`double` — Double

`boolean` — Boolean

`String`

array sorting

```
int A[]={5,7,1,3,9,2,4};
Arrays.sort(A);
for(int i=0;i<A.length;i++){
    System.out.print(A[i]+" ");
}
```

1) add(val) $\leftarrow$ to add the value at the end.

2) add(index, val) $\leftarrow$ add the value at the Particular index.

3) A[2] $\Rightarrow$ get(index) $\leftarrow$ to get the Particular index Value

4) size() $\rightarrow$ to get the length of array List.

5) remove(index) $\rightarrow$ to remove the Particular index element

6) set(index, val) $\rightarrow$ to update the value at Particular index.

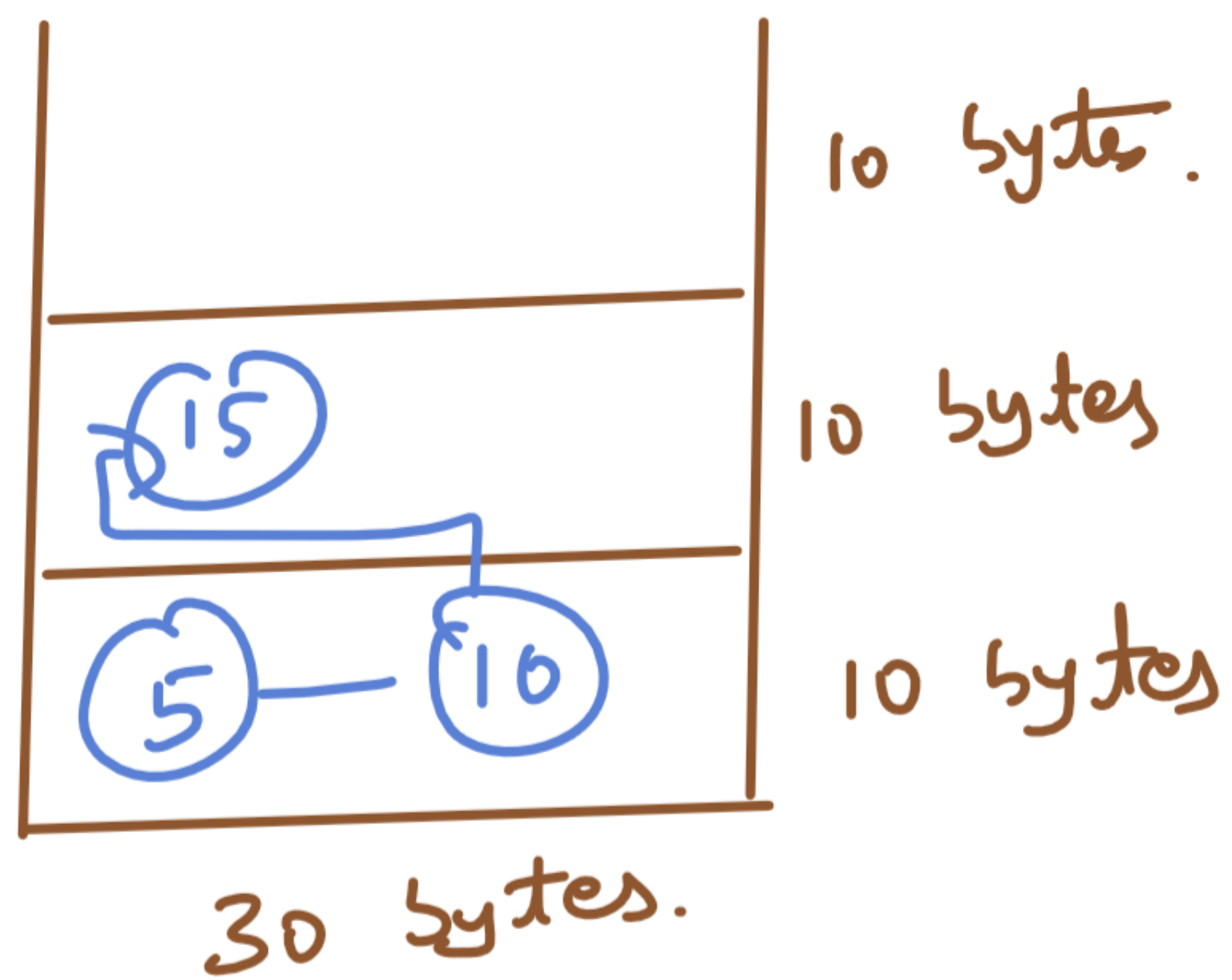
7) indexOf(^{val})
lastIndexOf(^{val}) $\rightarrow$ if element is Present it will return their index, else return -1.

8) clear() $\rightarrow$ to remove all elements.

```
ArrayList<Integer> al=new ArrayList<>();
al.add(10);
al.add(20);
System.out.println(al.size()); //2
al.add(30);
al.add(1,15);
System.out.println(al); //10 15 20 30
System.out.println(al.get(2)); //20
System.out.println(al.size()); //4
al.remove(1); //15
System.out.println(al); //10 20 30
al.set(0,44);
System.out.println(al); //44 20 30
al.add(44); //44 20 30 44
```

```
System.out.println(al.indexOf(44)); //0
System.out.println(al.lastIndexOf(44)); //3
System.out.println(al.indexOf(48)); //-1
Collections.sort(al); // 20 30 44 44
System.out.println(al); // 20 30 44 44
Collections.reverse(al);
System.out.println(al); // 44 44 30 20
al.clear(); //[]
System.out.println(al); //[]
```

How To Reuse Continuous memory allocation?



A:

5	10	15
---	----	----

Linked List

`LinkedList<data-type> name = new LinkedList();`

Methods in Linked List

- `add()` • `addFirst()` • `addLast()` • `add(idx, val)`
- `get(idx)` • `set(idx, val)` • `size()` • `remove()` — removing 1st element
- `removeFirst()` • `removeLast()` • `poll()` — removing 1st element
- `peek()` — get 1st element • `getFirst()` • `getLast()`
- `indexOf()` • `lastIndexOf()`

```
LinkedList ll=new LinkedList();
ll.add(10);
ll.add(20);
System.out.println(ll.size()); //2
ll.add(30);
ll.add(1,15);
System.out.println(ll); //10 15 20 30
System.out.println(ll.get(2)); //20
System.out.println(ll.size()); //4
ll.remove(1); //15
System.out.println(ll); //10 20 30
ll.set(0,44);
System.out.println(ll); //44 20 30
ll.add(44); //44 20 30 44
System.out.println(ll.indexOf(44)); //0
System.out.println(ll.lastIndexOf(44)); //3
```

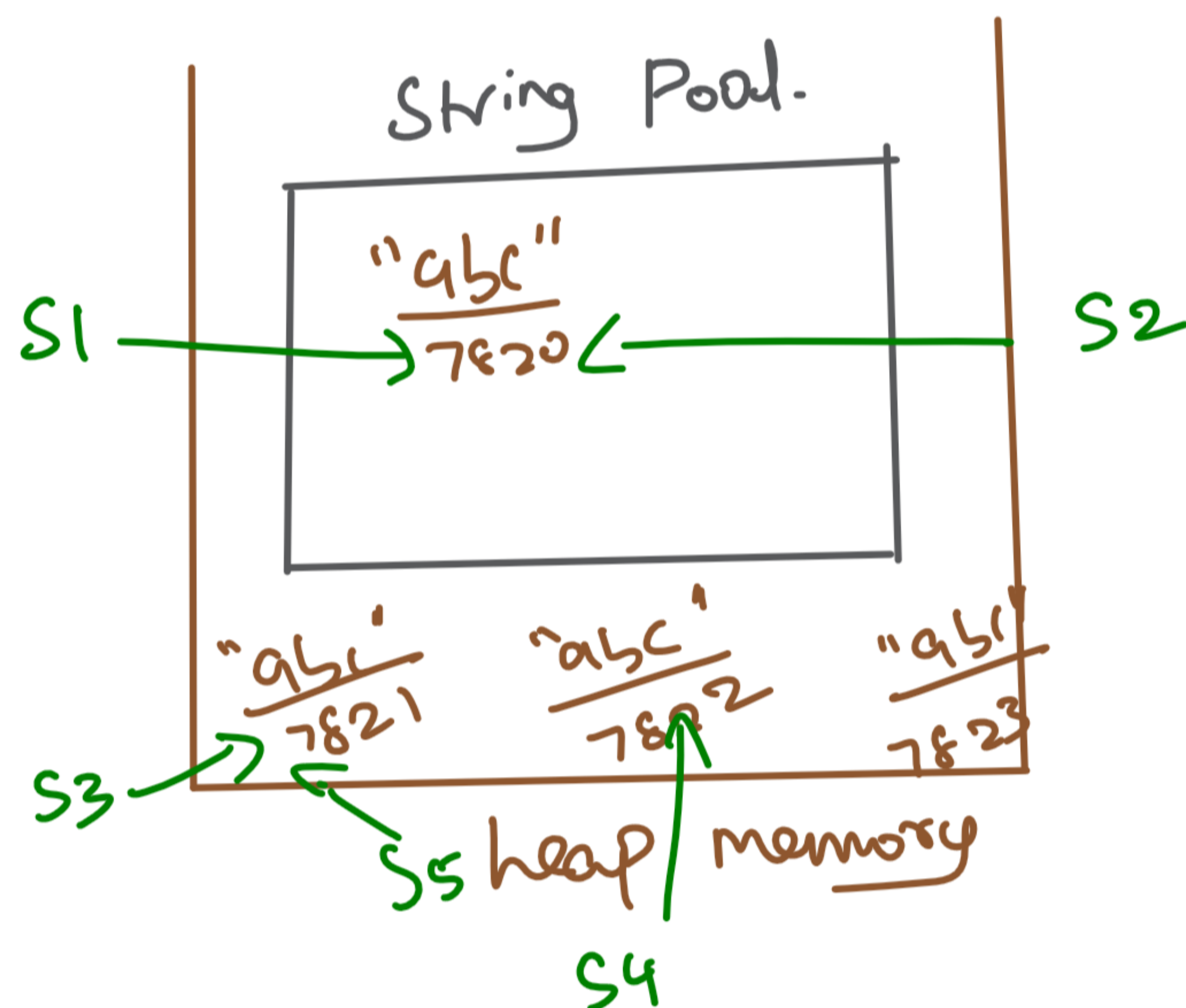
```
System.out.println(ll.indexOf(48)); //-1
Collections.sort(ll); // 20 30 44 44
System.out.println(ll); // 20 30 44 44
Collections.reverse(ll);
System.out.println(ll); // 44 44 30 20
ll.poll();
System.out.println(ll); //44 30 20
ll.clear(); //[]
System.out.println(ll); //[]
```

String

- it can be represented by double quotes (" ")
- sequence of or group of characters.
- string is an Immutable (we can not modify or change)

Initializing String

```
String s1="abc";  
String s2="abc";  
String s3=new String("abc");  
String s4=new String("abc");  
String s5=new String(s3);  
System.out.println(s4==s3?"Equal":"Not Equal");
```



- "==" is used to compare the addresses of the string.

String methods

- `s1.equals(s2)` → if both strings are equal it will return true else false.

```
String s1="abc";  
String s3=new String("abc");  
System.out.println(s1.equals(s3)?"Equal":"Not Equal");
```

- `s1.length()` → it will return the total no of characters in given string including white space.

```
String s1="I Love Python";  
System.out.println(s1.length()); //13
```

- `s1.charAt(idx)` $\leftarrow$ it will return the Particular Index character.

```
String s1="I Love Python";
System.out.println(s1.charAt(8)); //y
```

- `s1.substring(st, end)` $\leftarrow$ Part of the String, that should be continuous
 $\uparrow$ inc
 $\uparrow$ exc

```
String s1="I Love Python";
System.out.println(s1.substring(2,6));
/* Love*/
```

- `s1.substring(st)` $\leftarrow$ it will start from the st index, Print till end of the String.

```
String s1="I Love Python";
System.out.println(s1.substring(2));
/* Love Python */
```

- `s1.toLowerCase()` } it will return the new updated
`s1.toUpperCase()` } string, it won't modify the

```
String s1="I LOVE PYTHON"; original String.
String s2="i love python";
System.out.println(s1.toLowerCase());
System.out.println(s2.toUpperCase());
```

- `s1.replace(one-source, one-Target)` } one source $\rightarrow$ one Target

```
String s1="I Love Python, Python is Easy To Learn";
System.out.println(s1.replace("Python", "Java"));
```

```
I Love Java, Java is Easy To Learn
```

- `s1.replaceAll(many-source, one-Target)`

```
String s1="I Love Python, Python is Easy To Learn";
System.out.println(s1.replaceAll("[aeiouAEIOU]", "_"));
```

```
_ L _ v _ Pyth _ n, Pyth _ n _ s _ _ sy T _ L _ rn
```

- `s1.contains("source")` $\rightarrow$ if the given source is Present inside the string it will return true else false.

- `toCharArray()` → used to convert the String into char array.

Convert char Array into String

```
String s1="Python";
char Arr[]=s1.toCharArray();
//{'T','v','k','h','o','n'}
Arr[0]='T';
Arr[1]='v';
Arr[2]='k';
for(int i=0;i<Arr.length;i++){
    System.out.print(Arr[i]+" ");
}
String s2=new String(Arr);
String s3=String.valueOf(Arr);
System.out.println(s3);
```

- `s1.split("source")` ← it will split the String into string array based on source.

```
String s1="I Love Python";
String Arr[]=s1.split(" "); //{"I","Love","Python"}
System.out.println(Arr[0]+"--"+Arr[1]+"--");
```

- `s1.indexOf("source")`
`s1.lastIndexOf("source")` } if will return the index of given source, else return -1

```
String s1="I Love Python Love";
System.out.println(s1.indexOf("Love")); //2
System.out.println(s1.lastIndexOf("Love")); //14
```

StringBuilder or StringBuffer

Syntax:

`StringBuilder name = new StringBuilder();`

→ we can do all the string operation by using `StringBuilder`/`StringBuffer`. this is mutable. at the end this is not string.

```

StringBuilder sb=new StringBuilder("Apple");
sb.append(" Bus");//add the string at the end
System.out.println(sb);//Apple Bus
System.out.println(sb.length());//9
System.out.println(sb.charAt(4));//e
//sb.deleteCharAt(6);
//sb.delete(6,9);
sb.replace(6,9,"Banana");
//sb.insert(5,"Bus");
//sb.reverse();
System.out.println(sb.substring(3));
System.out.println(sb.toString());

```

```

Apple Bus
9
e
le Banana
Apple Banana

```

Hash Map

- Store the data in the form of Key Value Pair
- key is always unique.
- un-order map

```

HashMap<Integer,String> hm=new HashMap();
hm.put(2,"Jeeva");
//to put the key and value
hm.put(1,"Sangami");
hm.put(3,"Subha");
hm.put(2,"Jeevitha");
hm.putIfAbsent(3,"Santhosh");
System.out.println(hm);
System.out.println(hm.get(3));
//get the value based on key
System.out.println(hm.get(5));
// if key is not there , it return null
System.out.println(hm.getOrDefault(5,"Not Found"));
System.out.println(hm.size());
System.out.println(hm.containsKey(3));
System.out.println(hm.containsValue("Sangami"));
hm.put(4,"Abi");
System.out.println(hm);
hm.remove(4);
System.out.println(hm);
for(int val:hm.keySet()){
    System.out.print(val+" ");
}
System.out.println();
for(String val:hm.values()){
    System.out.print(val+" ");
}

```

```

{1=Sangami, 2=Jeevitha, 3=Subha}
Subha
null
Not Found
3
true
true
{1=Sangami, 2=Jeevitha, 3=Subha, 4=Abi}
{1=Sangami, 2=Jeevitha, 3=Subha}
1 2 3
Sangami Jeevitha Subha

```

Linked Hash Map → used to store the key value pair in the given order.

```
LinkedHashMap<Integer,Integer> hm=new LinkedHashMap();  
hm.put(7,70);  
hm.put(5,50);  
hm.put(1,10);  
hm.put(3,30);  
hm.put(2,20);  
System.out.println(hm);
```

{7=70, 5=50, 1=10, 3=30, 2=20}

Tree Map() → used to store the key value in either Ascending order or Descending order.

```
//TreeMap<Integer,Integer> hm=new TreeMap();  
TreeMap<Integer,Integer> hm=new TreeMap(Collections.reverseOrder());  
hm.put(7,70);  
hm.put(5,50);  
hm.put(1,10);  
hm.put(3,30);  
hm.put(2,20);  
System.out.println(hm);
```

{7=70, 5=50, 3=30, 2=20, 1=10}

HashSet()

→ we can not store duplicate values.

→ unordered

```
HashSet<Integer> hs=new HashSet();  
hs.add(34);  
hs.add(12);  
hs.add(5);  
hs.add(12);  
hs.remove(5);  
System.out.println(hs);  
System.out.println(hs.size());  
System.out.println(hs.contains(12));  
for(int val:hs){  
    System.out.print(val+" ");  
}
```

[34, 12]
2
true
34 12

1) LinkedHashSet() — To maintain given order

2) TreeSet() — make it Ascending or Descending.